

Punto 2.1 - Convenciones de modelado en Entity Framework Core

Ejercicio guiado: inspeccionar el modelo que EF Core construye para `AceriaData` y reconocer, antes de modificar nada, qué entidades, tablas, claves, propiedades, relaciones y comportamientos ha descubierto por convención.

Contexto del proyecto: este punto continúa exactamente desde `M01/PROYECTO/1.12`. El modelo heredado ya contiene `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`, el `AceriaDbContext`, SQL Server LocalDB y el contenedor de dependencias. En 2.1 no se rediseña el dominio ni se crea una base nueva: se observa el modelo existente para comprender qué ha deducido EF Core automáticamente. Esa lectura será la base de 2.2, donde comenzará la configuración explícita de propiedades.

Objetivos de aprendizaje

- Reconocer las convenciones que EF Core aplica al descubrir entidades.
- Identificar la tabla asociada a cada entidad.
- Comprobar cómo se detecta una clave primaria.
- Identificar claves foráneas y entidades principales.
- Interpretar la nulabilidad y las navegaciones a partir del modelo construido.
- Diferenciar lo que EF Core ha inferido por convención de lo que se configurará explícitamente en puntos posteriores.
- Resolver el `AceriaDbContext` mediante un ámbito local de DI, sin conservar un `ServiceProvider` estático.
- Verificar que una inspección de metadatos no necesita una migración nueva.

Paso 1: Abrir la solución autónoma de 2.1

Trabaja directamente sobre el estado completo de este punto:

```
cd M02/PROYECTO/2.1
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

La solución está dentro de la carpeta del punto. No es necesario abrir una solución global del repositorio ni copiar archivos desde `M1: 2.1` ya contiene el estado heredado que debe inspeccionarse.

Paso 2: Reconocer el estado heredado antes de tocar el modelo

Abre `Program.cs` y localiza las cuatro entidades y el `AceriaDbContext`. Antes de continuar, comprueba estas ideas:

- `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden` ya existen.
- `AceriaDbContext` expone los `DbSet<T>` correspondientes.
- `PlanchaAcero` contiene `OrdenId` y la navegación `Orden`.
- `OrdenFabricacion` contiene la colección `Planchas`.
- 2.1 no introduce todavía `DetalleOrden`, `CertificadoCalidad` ni `OrdenAleacion`.

El objetivo de este paso es separar claramente **observación** de **configuración**. Si se añade una relación nueva aquí, se estaría adelantando contenido de 2.4 o 2.5.

Paso 3: Resolver el DbContext con un ámbito local

Para evitar estado global innecesario, el proveedor y el ámbito se mantienen dentro de `Main`:

```
using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

Qué hace cada línea:

1. `BuildServiceProvider(...)` construye el contenedor después de registrar `AceriaDbContext`, repositorio y servicio.
2. `ValidateScopes = true` ayuda a detectar usos incorrectos de servicios `Scoped`.
3. `ValidateOnBuild = true` comprueba el grafo de dependencias al construir el proveedor.
4. `CreateScope()` crea el ámbito dentro del cual se resolverá el `DbContext`.
5. `GetRequiredService<AceriaDbContext>()` obtiene el contexto configurado para SQL Server LocalDB.
6. Los dos `using var` garantizan que proveedor y ámbito se liberen correctamente al finalizar.

No se necesita un campo `static IServiceProvider`. La inspección del modelo puede hacerse con el mismo patrón local que seguirá usando el curso.

| Paso 4: Enumerar entidades, tablas y claves primarias

A continuación se consulta **el modelo que EF Core ya ha construido**, no las clases mediante reflexión:

```
global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    var pk = entity.FindPrimaryKey();

    global::System.Console.WriteLine(
        $"Entidad: {entity.ClrType.Name} | " +
        $"Tabla: {entity.GetTableName()} | " +
        $"PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");
}
```

Observa especialmente:

- `context.Model.GetEntityTypes()` devuelve las entidades que forman parte del modelo EF Core.
- `GetTableName()` muestra la tabla relacional asociada.
- `FindPrimaryKey()` devuelve la clave primaria que EF Core reconoce.
- En `AceriaData`, las entidades usan `Id`, por lo que cumplen la convención de clave primaria.
- La convención general relevante es `Id` o `<NombreDelTipo>Id`; no depende del nombre del `DbSet`.

| Paso 5: Inspeccionar las claves foráneas descubiertas

El estado final de 2.1 también recorre las claves foráneas:

```
foreach (var fk in entity.GetForeignKeys())
{
    global::System.Console.WriteLine(
        $"FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> " +
        $"{fk.PrincipalEntityType.ClrType.Name}");
}
```

En `PlanchaAcero`, EF Core puede asociar `OrdenId` con la navegación `Orden` y con `OrdenFabricacion`. El ejercicio no crea esa relación en este punto: simplemente comprueba la relación que ya existía en el modelo heredado.

Esta diferencia es importante:

- **Descubrir una relación existente** pertenece a 2.1.
- **Configurar explícitamente una relación uno-a-muchos** pertenece a 2.3.
- **Crear relaciones uno-a-uno** pertenece a 2.4.
- **Modelar muchos-a-muchos con `OrdenAleacion`** pertenece a 2.5.

Paso 6: Compilar y ejecutar la inspección

Ejecuta el estado completo:

```
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

La salida debe contener la cabecera:

```
--- MODELO EF CORE 2.1 ---
```

y, a continuación, una línea por entidad con su tabla y su clave primaria. Para las entidades que tengan claves foráneas aparecerán además las líneas `FK:`.

No memorices la salida: úsala para contrastar lo que EF Core ha construido con lo que esperabas a partir de las clases.

Paso 7: Interpretar las convenciones observadas

Con la salida delante, comprueba una a una las convenciones relevantes.

Entidades y tablas. Las entidades incluidas en el modelo se mapean a las tablas determinadas por el modelo relacional y los `DbSet<T>` heredados.

Claves primarias. Una propiedad `Id` cumple la convención de clave primaria. También sería válida una propiedad con el patrón `<NombreDelTipo>Id`.

Claves foráneas. Una propiedad como `OrdenId`, combinada con la navegación `Orden`, permite a EF Core reconocer la relación con `OrdenFabricacion`.

Navegaciones. La referencia `PlanchaAcero.Orden` y la colección `OrdenFabricacion.Planchas` describen los dos extremos de la relación existente.

Nulabilidad. EF Core utiliza la información del tipo CLR y la configuración de nulabilidad de C# para construir la obligatoriedad del modelo. En 2.2 se trabajará esta parte de forma explícita.

Paso 8: Confirmar que inspeccionar el modelo no crea una migración

2.1 no cambia el esquema. Compruébalo con:

```
dotnet ef migrations list
```

Debe aparecer el historial heredado de M1 y **no** una migración nueva de 2.1.

Tampoco se usa `EnsureCreated()`. El curso mantiene el esquema gobernado por Migrations; inspeccionar metadatos no justifica recrear ni sustituir el historial existente.

Paso 9: Diagnosticar los errores más frecuentes

Antes de continuar, revisa qué ocurriría en cada uno de estos casos:

Situación	Qué problema provoca	Corrección
Guardar el <code>ServiceProvider</code> en un campo estático	Introduce estado global innecesario y oculta el ciclo de vida de los servicios	Mantener <code>provider</code> y <code>scope</code> locales dentro de <code>Main</code>
Eliminar o renombrar <code>Id</code> sin configurar otra clave	EF Core deja de poder determinar la PK de la entidad	Mantener <code>Id</code> o configurar la clave explícitamente en el punto correspondiente
Ejecutar <code>EnsureCreated()</code> para “ver si funciona”	Se sale del flujo basado en Migrations	Conservar el historial y limitar 2.1 a inspeccionar el modelo
Añadir una relación muchos-a-muchos	Adelanta contenido de 2.5	No modificar el dominio en 2.1
Interpretar el nombre del <code>DbSet</code> como regla de clave primaria	Confunde dos convenciones distintas	La PK se descubre por <code>Id</code> o <code><NombreDelTipo>Id</code>

Si quieres comprobar el error de clave primaria, hazlo únicamente sobre una copia desechable del ejercicio y restaura después el estado original. El checkpoint entregado debe permanecer válido.

Paso 10: Resolver el reto de inspección avanzada

Reto: ampliar la inspección para mostrar también el `DeleteBehavior` de cada clave foránea, sin cambiar el modelo.

El propio checkpoint contiene un bloque pedagógico comentado que puede activarse para realizar la prueba:

```
global::System.Console.WriteLine("--- RETO 2.1: DELETE BEHAVIOR ---");

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    foreach (var fk in entity.GetForeignKeys())
    {
        global::System.Console.WriteLine(
            $"FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> " +
            $"{fk.PrincipalEntityType.ClrType.Name} | DeleteBehavior: {fk.DeleteBehavior}");
    }
}
```

Este reto sigue perteneciendo a la inspección de metadatos: no añade tablas, columnas ni relaciones. La finalidad es aprender a leer con más detalle la relación que EF Core ya conoce.

Errores comunes

- Confundir inspeccionar el modelo con modificarlo.
- Copiar una versión antigua que conserva el proveedor de servicios en un campo estático.
- Usar `EnsureCreated()` o `EnsureDeleted()` como sustituto del historial de migraciones.
- Adelantar relaciones que corresponden a 2.4 o 2.5.
- Suponer que la clave primaria se deduce a partir del nombre del `DbSet`.
- Mirar sólo las clases y no contrastarlas con `context.Model`.

Analogía final

Antes de modificar una línea de producción, un técnico revisa los planos y recorre la instalación para saber qué máquinas existen, cómo están conectadas y qué función cumple cada unión. En 2.1 ocurre lo mismo: las clases son el diseño escrito, pero `context.Model` es el plano que EF Core ha construido realmente. Primero se aprende a leer ese plano; después, en los puntos siguientes, se empezará a modificarlo de forma consciente.

| Resultado esperado

Al finalizar 2.1 debes poder abrir el estado autónomo, compilarlo y ejecutar la inspección sin alterar el esquema. La salida debe identificar las entidades del modelo, sus tablas, las claves primarias y las claves foráneas existentes. `dotnet ef migrations list` debe seguir mostrando únicamente el historial heredado y el reto debe permitir consultar `DeleteBehavior` sin introducir una migración nueva.

| Conexión con el siguiente punto

Ya sabes distinguir lo que EF Core descubre automáticamente de lo que todavía no se ha configurado de forma explícita. En 2.2 partirás de este mismo modelo para trabajar con propiedades de negocio: requeridos, opcionales, longitudes máximas, precisión decimal y valores por defecto. El proyecto no se reinicia y las decisiones válidas de 2.1 se conservan.

| Código acumulativo completo del estado 2.1

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
}

```

```

    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
        _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOordenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");
        foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
        {
            var pk = entity.FindPrimaryKey();
            global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla:
            {entity.GetTableName()} | PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ??

```

```

Array.Empty<string>()))");
        foreach (var fk in entity.GetForeignKeys())
        {
            global::System.Console.WriteLine($" FK: {string.Join(", ", fk.Properties.Select(x =>
x.Name))} -> {fk.PrincipalEntityType.ClrType.Name}");
        }
    }
}
}

```

Explicación línea a línea del código acumulativo 2.1

Las referencias siguientes usan la numeración real de M02/PROYECTO/2.1/Program.cs. Se explican todas las líneas no vacías para que el documento sea trazable con el archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para los tipos o métodos usados en el archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `public class OrdenFabricacion` → declara la clase OrdenFabricacion que forma parte del estado acumulativo del ejercicio.

Línea 11: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 12: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 13: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad NumeroOrden de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 14: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad Cliente de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 15: `public DateTime FechaCreacion { get; set; }` → declara la propiedad FechaCreacion de tipo DateTime que EF Core o la lógica de negocio utiliza en este modelo.

Línea 16: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo List que EF Core o la lógica de negocio utiliza en este modelo.

Línea 17: `}` → cierra el bloque de código actual.

Línea 19: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo del ejercicio.

Línea 20: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 21: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 22: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 23: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 24: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 25: `public double Largo { get; set; }` → declara la propiedad Largo de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 26: `public OrdenFabricacion Orden { get; set; } = null!`; → declara la propiedad Orden de tipo OrdenFabricacion que EF Core o la lógica de negocio utiliza en este modelo.

Línea 27: `}` → cierra el bloque de código actual.

Línea 29: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo del ejercicio.

Línea 30: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 31: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 32: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 33: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 34: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double que EF Core o la lógica de negocio utiliza en este modelo.

Línea 35: `}` → cierra el bloque de código actual.

Línea 37: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo del ejercicio.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `public int Id { get; set; }` → declara la propiedad Id de tipo int que EF Core o la lógica de negocio utiliza en este modelo.

Línea 40: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 41: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string que EF Core o la lógica de negocio utiliza en este modelo.

Línea 42: `}` → cierra el bloque de código actual.

Línea 44: `public class AceriaDbContext : DbContext` → declara la clase AceriaDbContext que forma parte del estado acumulativo del ejercicio.

Línea 45: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 46: `public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → declara el constructor o método AceriaDbContext con los parámetros necesarios para este componente.

Línea 48: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 49: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 50: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 51: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultar y persistir mediante EF Core.

Línea 53: `}` → cierra el bloque de código actual.

Línea 55: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` para desacoplar consumidores e implementación.

Línea 56: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 57: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 58: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 59: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 60: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 61: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 62: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 63: `}` → cierra el bloque de código actual.

Línea 65: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo del ejercicio.

Línea 66: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 67: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 68: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → declara el constructor o método `OrdenRepositorio` con los parámetros necesarios para este componente.

Línea 70: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado de la consulta antes de materializarlo.

Línea 71: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 72: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null cuando no existe.

Línea 73: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad o relación nueva en el `ChangeTracker` para insertarla al guardar.

Línea 74: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca la entidad indicada para borrado al confirmar los cambios.

Línea 75: `public int Guardar() => _context.SaveChanges();` → persiste en SQL Server los cambios seguidos por el `DbContext`.

Línea 76: `}` → cierra el bloque de código actual.

Línea 78: `public interface IServicioOrdenes` → declara el contrato `IServicioOrdenes` para desacoplar consumidores e implementación.

Línea 79: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 80: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 81: `}` → cierra el bloque de código actual.

Línea 83: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase `ServicioOrdenes` que forma parte del estado acumulativo del ejercicio.

Línea 84: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 85: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado `_repositorio` de tipo `IOrdenRepositorio` que conserva una dependencia de la clase.

Línea 86: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext` que conserva una dependencia de la clase.

Línea 87: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor o método `ServicioOrdenes` con los parámetros necesarios para este componente.

Línea 88: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 89: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 90: `_context = context;` → guarda el `DbContext` inyectado para consultar los datos necesarios durante el servicio.

Línea 91: `}` → cierra el bloque de código actual.

Línea 93: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 94: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 95: `var orden = _repositorio.ObtenerPorId(ordenId);` → consulta la orden mediante el repositorio y conserva el resultado anulable en una variable local.

Línea 96: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa la condición y ejecuta el bloque asociado sólo cuando se cumple.

Línea 97: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta construida.

Línea 98: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado calculado al código que invocó el método.

Línea 99: `}` → cierra el bloque de código actual.

Línea 100: `}` → cierra el bloque de código actual.

Línea 102: `public static class Program` → declara la clase `Program` que forma parte del estado acumulativo del ejercicio.

Línea 103: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 104: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 105: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 106: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 107: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base usada para localizar los archivos de configuración.

Línea 108: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora `appsettings.json` a la configuración.

Línea 109: `.AddEnvironmentVariables()` → incorpora variables de entorno como fuente adicional de configuración.

Línea 110: `.Build();` → materializa la configuración o el objeto encadenado construido en las líneas anteriores.

Línea 112: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena de conexión `AceriaDB` desde la configuración.

Línea 113: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene `ConnectionStrings:AceriaDB`.

Línea 115: `var services = new ServiceCollection();` → crea la colección donde se registran las dependencias de la aplicación.

Línea 116: `services.AddDbContext<AceriaDbContext>(options =>` → registra el `DbContext` en el contenedor de inyección de dependencias.

Línea 117: `options` → continúa la configuración del `DbContext` sobre el parámetro `options` recibido por la lambda de `AddDbContext`.

Línea 118: `.UseSqlServer(connectionString, sql =>` → selecciona `SQL Server` como proveedor de `EF Core`.

Línea 119: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 120: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios del proveedor `SQL Server`.

Línea 121: `sql.CommandTimeout(60);` → establece el tiempo máximo permitido para los comandos `SQL`.

Línea 122: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 123: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura la salida del logging de `EF Core`.

Línea 124: `.EnableDetailedErrors();` → activa errores detallados para facilitar el diagnóstico del laboratorio.

Línea 126: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 127: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio `scoped` para compartir dependencias dentro del mismo ámbito.

Línea 129: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el `ServiceProvider` con validación de servicios y garantiza su liberación al salir del ámbito mediante `using var`.

Línea 130: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 131: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias `scoped` usadas desde ámbitos incorrectos.

Línea 132: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el `ServiceProvider`.

Línea 133: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 135: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 136: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio desde el contenedor de dependencias.

Línea 137: `global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 138: `foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))` → ordena el resultado de la consulta antes de materializarlo.

Línea 139: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 140: `var pk = entity.FindPrimaryKey();` → consulta los metadatos de EF Core para obtener la clave primaria de la entidad.

Línea 141: `global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 142: `foreach (var fk in entity.GetForeignKeys())` → enumera las claves foráneas configuradas para la entidad.

Línea 143: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 144: `global::System.Console.WriteLine($" FK: {string.Join(", ", fk.Properties.Select(x => x.Name))} -> {fk.PrincipalEntityType.ClrType.Name}");` → escribe en consola la evidencia usada para verificar el comportamiento del estado.

Línea 145: `}` → cierra el bloque de código actual.

Línea 146: `}` → cierra el bloque de código actual.

Línea 147: `}` → cierra el bloque de código actual.

Línea 148: `}` → cierra el bloque de código actual.